

SENIN 16 MARET 2026

LAPORAN HASIL PRAKTIKUM

Jobsheet 05



Disusun oleh:

Rafif Rizdan Prastana

Kelas 1H/TI

254107020052

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

5.1 Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

1. Mahasiswa mampu membuat algoritma bruteforce dan divide-conquer
2. Mahasiswa mampu menerapkan penggunaan algoritma bruteforce dan divide-conquer

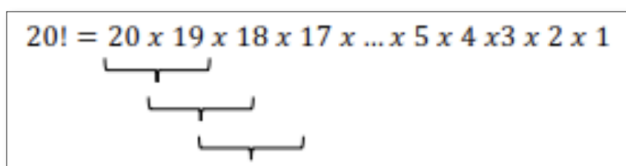
5.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

Perhatikan Diagram Class berikut ini:

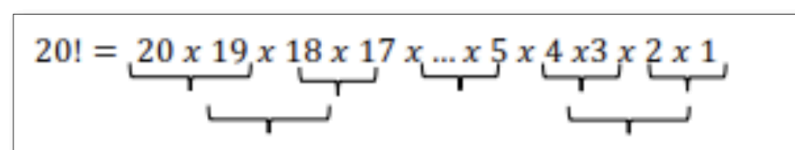
Faktorial
faktorialBF(): int faktorialDC(): int

Berdasarkan diagram class di atas, akan dibuat program class dalam Java. Untuk menghitung nilai faktorial suatu angka menggunakan 2 jenis algoritma, Brute Force dan Divide and Conquer. Jika digambarkan terdapat perbedaan proses perhitungan 2 jenis algoritma tersebut sebagai berikut :

Tahapan pencarian nilai faktorial dengan algoritma Brute Force:


$$20! = 20 \times 19 \times 18 \times 17 \times \dots \times 5 \times 4 \times 3 \times 2 \times 1$$

Tahapan pencarian nilai faktorial dengan algoritma Divide and Conquer:


$$20! = 20 \times 19 \times 18 \times 17 \times \dots \times 5 \times 4 \times 3 \times 2 \times 1$$

5.2.1. Langkah-langkah Percobaan

1. Buat Project baru, dengan nama “**BruteForceDivideConquer**” Buat package dengan nama minggu5.
2. Buatlah class baru dengan nama Faktorial
3. Lengkapi class Faktorial dengan atribut dan method yang telah digambarkan di dalam diagram class di atas, sebagai berikut:

- a) Tambahkan method faktorialBF():

```
1 public class Faktorial {
2
3     int faktorialBF(int n){
4         int fakto = 1;
5         for(int i=1; i<=n; i++){
6             fakto = fakto * i;
7         }
8         return fakto;
9     }
```

- b) Tambahkan method faktorialDC():

```
1 int faktorialDC(int n){
2     if(n==1){
3         return 1;
4     }else{
5         int fakto = n * faktorialDC(n-1);
6         return fakto;
7     }
8 }
```

4. Coba jalankan (Run) class Faktorial

- a) dengan membuat class baru MainFaktorial. Di dalam fungsi main sediakan komunikasi dengan user untuk memasukkan nilai yang akan dicari faktorialnya

```
1 Scanner input = new Scanner(System.in);
2 System.out.print("Masukkan nilai: ");
3 int nilai = input.nextInt();
```

- b) Kemudian buat objek dari class Faktorial dan tampilkan hasil pemanggilan method

```
1 Faktorial fk = new Faktorial();
2 System.out.println("Nilai faktorial " + nilai +
3     " menggunakan BF: " + fk.faktorialBF(nilai));
4 System.out.println("Nilai faktorial " + nilai +
5     " menggunakan DC: " + fk.faktorialDC(nilai));
```

- c) Pastikan program sudah berjalan dengan baik!

5.2.2. Verifikasi Hasil Percobaan

Cocokkan hasil compile kode program anda dengan gambar berikut ini.

```
DanStukturData_9ad25eaa/bin MainFaktorial
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```

5.2.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!
3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !
4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

5.2.3 Jawaban

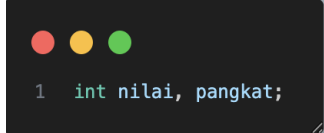
1. If adalah titik berhenti, else adalah bagian yang “membagi” masalah.
2. Bisa, menggunakan do-while
3. Fakto *=i; adalah iteratif (perulangan), fakto = n * faktorialDC(n-1); adalah rekursif atau memanggil diri sendiri
4. Jika faktorialBF() menggunakan perulangan for yang berjalan dari i=1 sampai i=n, sedangkan faktorialDC()! memecah masalah menjadi sub-masalah yang lebih kecil(n-1) hingga mencapai base case (n==1)

5.3 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

Pada praktikum ini kita akan membuat program class dalam Java, untuk menghitung nilai pangkat suatu angka menggunakan 2 jenis algoritma, Brute Force dan Divide and Conquer. Pada praktikum ini akan digunakan Array of Object untuk mengelola beberapa objek yang akan dibuat, berbeda dengan praktikum tahap sebelumnya yang hanya berfokus pada 1 objek factorial saja.

5.3.1. Langkah-langkah Percobaan

1. Di dalam paket minggu5, buatlah class baru dengan nama Pangkat. Dan di dalam class Pangkat tersebut, buat atribut angka yang akan dipangkatkan sekaligus dengan angka pemangkatnya



```
1 int nilai, pangkat;
```

2. Tambahkan konstruktor berparameter



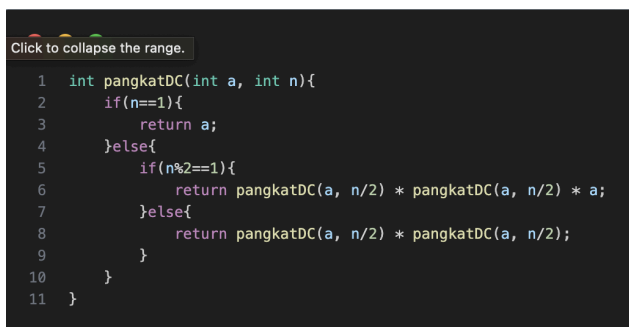
```
1 Pangkat(int n, int p){
2     nilai = n;
3     pangkat = p;
4 }
```

3. Pada class Pangkat tersebut, tambahkan method PangkatBF()



```
1 int pangkatBF(int a, int n){
2     int hasil = 1;
3     for(int i=0; i<n; i++){
4         hasil = hasil * a;
5     }
6     return hasil;
7 }
```

4. Pada class Pangkat juga tambahkan method PangkatDC()



```
Click to collapse the range.
1 int pangkatDC(int a, int n){
2     if(n==1){
3         return a;
4     }else{
5         if(n%2==1){
6             return pangkatDC(a, n/2) * pangkatDC(a, n/2) * a;
7         }else{
8             return pangkatDC(a, n/2) * pangkatDC(a, n/2);
9         }
10    }
11 }
```

5. Perhatikan apakah sudah tidak ada kesalahan yang muncul dalam pembuatan class Pangka
6. Selanjutnya buat class baru yang di dalamnya terdapat method main. Class tersebut dapat dinamakan MainPangkat. Tambahkan kode pada class main untuk menginputkan jumlah elemen yang akan dihitung pangkatnya.



```
1 Scanner input = new Scanner(System.in);
2 System.out.print("Masukkan jumlah elemen: ");
3 int elemen = input.nextInt();
```

7. Nilai pada tahap 5 selanjutnya digunakan untuk instansiasi array of objek. Di dalam Kode berikut ditambahkan proses pengisian beberapa nilai yang akan dipangkatkan sekaligus dengan pemangkatnya.

```
1  Pangkat[] png = new Pangkat[elemen];
2  for(int i=0; i<elemen; i++){
3      System.out.print("Masukan nilai basis elemen ke-"+(i+1)+" : ");
4      int basis = input.nextInt();
5      System.out.print("Masukan nilai pangkat elemen ke-"+(i+1)+" : ");
6      int pangkat = input.nextInt();
7      png[i] = new Pangkat(basis, pangkat);
8  }
```

8. Kemudian, panggil hasil nya dengan mengeluarkan return value dari method PangkatBF() dan PangkatDC().

```
1  System.out.println("HASIL PANGKAT BRUTEFORCE:");
2  for(Pangkat p : png){
3      System.out.println(p.nilai+"^"+p.pangkat+" : "+p.pangkatBF(p.nilai, p.pangkat));
4  }
5
6  System.out.println("HASIL PANGKAT DIVIDE AND CONQUER:");
7  for(Pangkat p : png){
8      System.out.println(p.nilai+"^"+p.pangkat+" : "+p.pangkatDC(p.nilai, p.pangkat));
9  }
```

5.3.2. Verifikasi Hasil Percobaan

Pastikan output yang ditampilkan sudah benar seperti di bawah ini.

```
DanStukturData_9ad25eaa/bin MainPangkat
Masukkan jumlah elemen: 3
Masukan nilai basis elemen ke-1: 2
Masukan nilai pangkat elemen ke-1: 3
Masukan nilai basis elemen ke-2: 4
Masukan nilai pangkat elemen ke-2: 5
Masukan nilai basis elemen ke-3: 6
Masukan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```

5.3.3. Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!
2. Apakah tahap combine sudah termasuk dalam kode tersebut? Tunjukkan!
3. Pada method pangkatBF() terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?
4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!

5.3.3 Jawaban

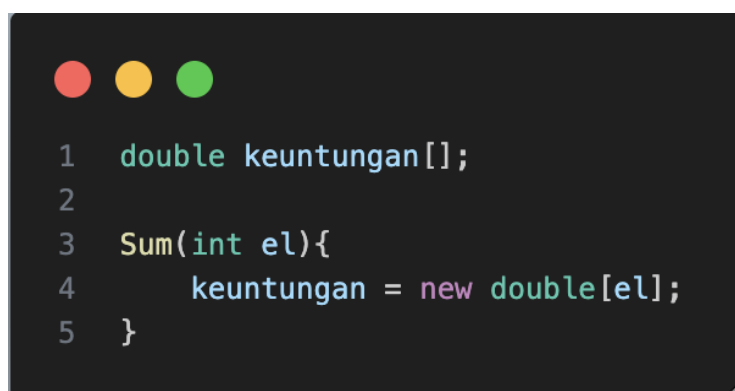
1. Jika pangkatBF() kondisi berhenti saat loop berhenti saat $i \geq n$, sedangkan pangkatDC()! kondisi berhenti jika base case saat $n == 1$, return a.
2. Ya, ada di method pangkatDC()
3. Pemanggilan di main menjadi singkat. sebelum dengan parameter (**p.pangkatBF(p.nilai,p.pangkat)**), sedangkan tanpa parameter (**p.pangkatBF()**). keduanya valid tpi tanpa parameter lebih ringkas
4. PangkatBF() menghitung dengan cara mengalikan a sebanyak n kali secara berurutan dalam bentuk for. sedangkan pangkatDC() menghitung dengan cara membagi pangkat menjadi setengahnya secara rekursif, lalu menggabungkan hasilnya.

5.4 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

Di dalam percobaan ini, kita akan mempraktekkan bagaimana proses divide, conquer, dan combine diterapkan pada studi kasus penjumlahan keuntungan suatu perusahaan dalam beberapa bulan.

5.4.1. Langkah-langkah Percobaan

1. Pada paket minggu 5. Buat class baru yaitu class **Sum**. Tambahkan pula konstruktor pada class Sum.



```
1  double keuntungan[];  
2  
3  Sum(int el){  
4      keuntungan = new double[el];  
5  }
```

2. Tambahkan method TotalBF() yang akan menghitung total nilai array dengan cara iterative.

```
1 double totalBF(){
2     double total = 0;
3     for(int i=0; i<keuntungan.length; i++){
4         total = total + keuntungan[i];
5     }
6     return total;
7 }
```

3. Tambahkan pula method TotalDC() untuk implementasi perhitungan nilai total array menggunakan algoritma Divide and Conquer

```
1 double totalDC(double arr[], int l, int r){
2     if(l==r){
3         return arr[l];
4     }
5
6     int mid = (l+r)/2;
7     double lsum = totalDC(arr, l, mid);
8     double rsum = totalDC(arr, mid+1, r);
9     return lsum + rsum;
10 }
```

4. Buat class baru yaitu MainSum. Di dalam kelas ini terdapat method main. Pada method ini user dapat menuliskan berapa bulan keuntungan yang akan dihitung. Dalam kelas ini sekaligus dibuat instansiasi objek untuk memanggil atribut ataupun fungsi pada class Sum

```
1 Scanner input = new Scanner(System.in);
2 System.out.print("Masukkan jumlah elemen: ");
3 int elemen = input.nextInt();
```

5. Buat objek dari class Sum. Lakukan perulangan untuk mengambil input nilai keuntungan dan masukkan ke atribut keuntungan dari objek yang baru dibuat tersebut!

```
1 Sum sm = new Sum(elemen);
2 for(int i=0; i<elemen; i++){
3     System.out.print("Masukkan keuntungan ke-"+(i+1)+" : ");
4     sm.keuntungan[i] = input.nextDouble();
5 }
```


6. Tampilkan hasil perhitungan melalui objek yang telah dibuat untuk kedua cara yang ada (Brute Force dan Divide and Conquer)

```
1 System.out.println("Total keuntungan menggunakan Bruteforce: "+sm.totalBF());
2 System.out.println("Total keuntungan menggunakan Divide and Conquer: "+sm.totalDC(sm.keuntungan, 0, elemen-1));
```

5.4.2. Verifikasi Hasil Percobaan

Cocokkan hasil compile kode program anda dengan gambar berikut ini.

```
DanStukturData_9ad25eaa/bin MainSum
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```

5.4.3. Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?
2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

4. Apakah base case dari totalDC()?
5. Tarik Kesimpulan tentang cara kerja totalDC()

5.4.3 Jawaban

1. Dengan menggunakan mid, array dibagi menjadi 2 bagian kiri indeks =1 sampai mid sedangkan bagian kanan indeks mid+1 sampai r. tanpa mid, tidak ada cara untuk mengetahui dimana array harus dipotong menjadi dua bagian.

2. lsum untuk menjumlahkan semua elemen di bagian kiri (dari 1 hingga mid), sedangkan rsum menjumlahkan semua elemen bagian kanan array (dari mid+1 hingga r)
3. Keduanya harus digabungkan agar menghasilkan total keseluruhan array. Tanpa return lsum+rsum; hasil dari bagian kiri dan kanan tidak akan pernah bergabung menjadi total yang utuh.

4.



```

1  if(l==r){
2      return arr[l];
3  }

```

5. Menghitung total array dengan cara membagi array menjadi dua bagian secara rekursif, menghitung jumlah masing-masing bagian, lalu menggabungkan hasilnya. Prosesnya :
 - a. Divide = Array dipotong di tengah menggunakan **mid**
 - b. Conquer = Setiap bagian dijumlahkan secara rekursif hingga tersisa 1 elemen
 - c. Combine = Hasil kiri dan kanan dijumlahkan dengan **lsum + rsum**

Proses terus berulang hingga mencapai base case (**l==r**), yaitu saat array tidak bisa dibagi lagi dan langsung mengembalikan nilai elemen tersebut.

4.5 Latihan Praktikum



```

1  public class MainNilai {
2
3      public static void main(String[] args) {
4          int[] nilaiUTS = {85, 90, 78, 92, 88};
5          int[] nilaiUAS = {80, 85, 75, 95, 90};
6
7          int utsTertinggi = Nilai.cariMaksUTS(nilaiUTS, 0, nilaiUTS.length - 1);
8          int utsTerendah = Nilai.cariMinUTS(nilaiUTS, 0, nilaiUTS.length - 1);
9          double rataUAS = Nilai.hitungRataUAS(nilaiUAS);
10
11          System.out.println("Nilai UTS Tertinggi: " + utsTertinggi);
12          System.out.println("Nilai UTS Terendah: " + utsTerendah);
13          System.out.println("Rata-rata Nilai UAS: " + rataUAS);
14      }
15  }

```

```

1  class Nilai {
2
3      public static int cariMaksUTS(int[] nilai, int kiri, int kanan) {
4          if (kiri == kanan) {
5              return nilai[kiri];
6          }
7
8          int tengah = (kiri + kanan) / 2;
9
10         int maksKiri = cariMaksUTS(nilai, kiri, tengah);
11         int maksKanan = cariMaksUTS(nilai, tengah + 1, kanan);
12
13         return Math.max(maksKiri, maksKanan);
14     }
15
16     public static int cariMinUTS(int[] nilai, int kiri, int kanan) {
17         if (kiri == kanan) {
18             return nilai[kiri];
19         }
20
21         int tengah = (kiri + kanan) / 2;
22
23         int minKiri = cariMinUTS(nilai, kiri, tengah);
24         int minKanan = cariMinUTS(nilai, tengah + 1, kanan);
25
26         return Math.min(minKiri, minKanan);
27     }
28
29     public static double hitungRataUAS(int[] nilai) {
30         int total = 0;
31
32         for (int i = 0; i < nilai.length; i++) {
33             total += nilai[i];
34         }
35
36         return (double) total / nilai.length;
37     }
38 }

```

Output :

```

DanStukturData_9ad25eaa/bin MainNilai
Nilai UTS Tertinggi: 92
Nilai UTS Terendah: 78
Rata-rata Nilai UAS: 85.0
❖ rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %

```